Market Simulation

From first principles to a production-oriented
event-driven market-making simulator

Saxon Lee

March 2026

Abstract

This document traces the full arc of the project I started last year. It starts with the v2.0 MVP - an order book, a quoting agent, and enough telemetry to see what is happening - and works through the conceptual models I built to understand what a realistic market environment actually needs: toxicity regimes, liquidity shocks, fat-tailed price moves, and the asymmetry between informed and uninformed flow. The second half explains why v2.0 would be binned by a practitioner at a serious firm, and how v3.0 works to fix it. The changes are all under the hood. They address CPU cache layout, realistic network latency, and the hard boundary between what a market maker can observe and what it cannot.

Contents

1 Purpose and audience **2**

2 The v2.0 MVP **2**

3 Conceptual groundwork: thinking about a realistic environment **3**
3.1 Hidden-Markov toxicity regimes . . . 3
3.2 Occasional order book sweeps . . . 3
3.3 Jump-diffusion mid . . . 4

4 Agent policy and maker/taker economics **4**
4.1 Quoting with inventory skew . . . 4
4.2 Urgent inventory flattening . . . 5
4.3 Fees, rebates, and net exchange P&L . . . 5

5 Risk measures and HFT diagnostics **5**
5.1 Equity and per-step P&L . . . 5
5.2 VaR and CVaR . . . 5
5.3 Drawdown and duration . . . 6
5.4 Rolling information ratio . . . 6
5.5 HFT-specific diagnostics . . . 6

6 The reality check: why v2.0 was a toy **6**

7 The v3.0 engine: production-oriented architecture **7**
7.1 OrderPool: slab allocator with intrusive lists . . . 7
7.2 ArrayBook: O(1) level access . . . 7
7.3 LatencyModel: log-normal tails and congestion . . . 8
7.4 The Hawkes flow model . . . 8

7.5 Informed traders and information asymmetry 8

8 The v3.0 agent: observable signals only **9**
8.1 FairValueModel 9
8.2 Toxicity-adaptive spread 9
8.3 ReplayEngine 9

9 Streamlit viewer (v3.0) **9**

10 Build and run **10**

11 Summary of changes from v2.0 to v3.0 **11**

12 What I learnt **11**

1 Purpose and audience

After some time around trading desks, and conversations with quants at the likes of Jump, Jane Street and Five Rings, I want to work as a quant researcher/trader. Before running I need to walk, so I built this to learn by doing rather than by reading. My aim was a small but honest loop: implement a quoting policy, watch every state change, measure what actually matters, and be honest when the model is wrong.

This document is written for two audiences simultaneously. A hiring manager should be able to read it and understand what the project is about. A practitioner should be able to read it and find no obvious hand-waving. Where there are simplifications I say so.

2 The v2.0 MVP

The starting point was a single C++ binary and two Python tools. The core loop was:

1. **Order book.** Price levels held in `std::map`, with a `std::vector` per level as the FIFO queue. Market orders walk the book. Cancels remove by order ID. All prices snap to an integer tick at ingress.

2. **Agent.** A symmetric market maker that quotes around a mid, skews by inventory, reduces size as risk builds, enforces a minimum quote life and a cancel latency, and flattens with taker orders if inventory crosses a threshold.

3. **Driver.** One step per iteration: place or maintain quotes, draw external flow, match, then write a single end-of-step snapshot to CSV.

The CSV columns were time, equity, inventory, bid, ask. A plotting script produced equity and drawdown curves, inventory, the BBO, a histogram of per-step P&L, and a rolling information ratio. The Streamlit viewer loaded the CSVs and let me compare runs.

That is it. No HMM, no Hawkes process, no informed traders. Just a book, an agent, and enough logging to see when things went wrong.

3 Conceptual groundwork: thinking about a realistic environment

A simulator that just flips a coin for the mid will not stress the right things. Fills look too benign and risk is understated. Before building the v3.0 engine I worked through three conceptual models - not all of which survived into the final implementation - that shaped how I thought about what a realistic environment needs to do.

3.1 Hidden-Markov toxicity regimes

What an HMM is

A hidden Markov model is a two-layer process. There is an unobserved state S_t evolving as a Markov chain, and observations Y_t whose distribution depends on S_t. For a market context, the hidden state represents regime: normal versus toxic. The chain has a transition matrix

$$\mathbf{P} = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}, \qquad \alpha, \beta \in [0, 1].$$

In state N the external flow rate is moderate and the directional bias is mild. In state T the rate is elevated and the bias is stronger. This makes flow cluster naturally without needing to hard-code bursts.

The expected run length in state N is $1/\alpha$ steps; in T it is $1/\beta$. Setting $\alpha = 0.05$ and $\beta = 0.20$ gives typical normal runs of 20 steps and typical toxic bursts of 5 steps, which felt roughly right for a high-frequency setting.

Why the intuition matters, even if the HMM was superseded

Real markets show toxic bursts: meta-orders, news arrivals, liquidity withdrawals. During a toxic burst a naive market maker earns fewer profitable fills, suffers more adverse selection, and faces larger drawdowns. The HMM was my early attempt to encode this. The 6-dimensional Hawkes process in v3.0 encodes the same intuition - burst clustering is now implicit in the self-excitation structure - but thinking through the HMM first made the design constraints clear.

What to measure when regimes are present

- Realised spread separated by regime. Does the strategy earn its target spread in both states?
- Adverse selection by regime. How much of the edge is given back to informed flow in toxic periods?
- Whether the inventory policy adapts fast enough. A fixed-spread strategy in a toxic burst accumulates a one-sided position faster than it can flatten.

3.2 Occasional order book sweeps

What I modelled

With small probability p_{sweep} at each step, a large aggressive order removes several consecutive price levels. The sweep depth D (number of levels removed) is drawn from a heavy-tailed distribution - a geometric or truncated power law - to capture the fact that most sweeps are shallow but rare deep sweeps are disproportionately painful.

Why it matters

Sweeps are the simplest way to model gap risk. If you carry inventory and a sweep runs through your side, the loss is proportional to both your position and the sweep depth. The interaction with cancel latency is the key lesson: if your cancel arrives after the sweep has started, your quote gets lifted at a price that is no longer informative.

The metrics I tracked on sweep steps were: loss conditional on the event, time to recover to the pre-sweep equity peak, and inventory h steps after the event. These three numbers together tell you whether the strategy can absorb the shock or whether it compounds.

3.3 Jump-diffusion mid

What it is

A discretised Merton jump-diffusion for the mid price:

$$m_t = m_{t-1} + \delta\,\varepsilon_t + \sum_{k=1}^{N_t} J_{t,k}, \qquad \varepsilon_t \in \{-1,+1\},\ \mathbb{P}(+1) = \tfrac{1}{2},$$

where $N_t \sim \text{Poisson}(\kappa)$ and jumps $J_{t,k} \sim \mathcal{N}(0, \sigma_J^2)$. The mid is snapped to the tick grid before quoting.

Why not a plain random walk

A one-tick random walk understates tail risk. It makes carrying inventory look cheap because the adverse move after a fill is bounded by one tick per step almost surely. Jumps generate honest fat-tailed P&L and force the agent to respect inventory limits in a way that a symmetric random walk never will. They also reveal whether urgent flattening earns its cost: if jumps cluster (via the Poisson mean κ being large), the agent pays taker fees frequently and they had better be buying back meaningful risk.

Parameter intuition.

- δ controls ordinary noise. With tick rounding, setting δ close to the tick size means the mid moves one tick most steps.
- κ is the average number of jumps per step. Start at 0.01 and increase gradually; the effect on tail risk is non-linear.
- σ_J sets jump magnitude in tick units. A few ticks is enough to force discipline on inventory limits.

4 Agent policy and maker/taker economics

4.1 Quoting with inventory skew

The reservation price is the fair value of the mid from the agent's perspective, adjusted for inventory:

$$r_t = \text{FairValue}_t - c_{\text{inv}} \cdot \text{inv}_t.$$

Quotes are placed at $r_t \pm s_t/2$ where the half-spread $s_t/2$ widens as $|\text{inv}_t|$ grows. This leans against accumulation: when the agent is long, the bid moves down (less eager to buy more) and the ask moves up (more competitive to sell). The effect on fill quality is a measurable improvement in markout on the inventory-reducing side.

4.2 Urgent inventory flattening

When $|\text{inv}_t|$ crosses a hard threshold, a taker market order cuts the position. This costs a fee but shortens drawdowns. The telemetry makes the trade-off visible: tail loss shrinks, but cumulative fees rise. Whether that is worth it depends on the fee tier and the magnitude of the jumps being hedged.

4.3 Fees, rebates, and net exchange P&L

Let f_M be the maker rebate per share (positive if the exchange pays for passive liquidity) and f_T the taker fee per share. Exchange economics over a run:

$$\Pi_{\text{exch}} = f_M \cdot Q^{\text{maker}} - f_T \cdot Q^{\text{taker}}.$$

Total strategy P&L is realised spread plus mark-outs plus Π_{exch}. A rough per-share break-even condition:

$$\mathbb{E}[\text{profit per share}] \approx \frac{s}{2} - \mathbb{E}[\text{adverse move}] + f_M - p_{\text{urgent}} \cdot f_T,$$

where p_{urgent} is the fraction of shares exited with taker orders. Many venues pay meaningful rebates for passive liquidity. If you do not track Π_{exch} separately you will misattribute where the money comes from and underestimate how sensitive the strategy is to changes in fee tiers.

5 Risk measures and HFT diagnostics

5.1 Equity and per-step P&L

End-of-step equity:

$$E_t = \text{cash}_t + (m_t - \bar{p}_t) \cdot \text{inv}_t,$$

where $\bar{p}_t$ is the average cost basis. Per-step P&L is $\Delta E_t = E_t - E_{t-1}$. The accounting identity $E_t = \text{cash}_t + \text{inv_value}_t$ should hold to machine precision at every step; the Streamlit viewer plots the residual as a diagnostic.

5.2 VaR and CVaR

Define step losses $L_t = -\Delta E_t$. For tail level α:

$$\begin{aligned} \text{VaR}_\alpha &= \alpha\text{-quantile of } L_t, \\ \text{CVaR}_\alpha &= \mathbb{E}[L_t \mid L_t \geq \text{VaR}_\alpha]. \end{aligned}$$

Compute these on per-step data for a granular view, and then on session-aggregated blocks to see daily tails. Comparing VaR and CVaR across normal versus toxic regimes tells you whether the strategy's tail risk is regime-stable.

5.3 Drawdown and duration

Let $P_t = \max_{u \le t} E_u$ be the running equity peak. Drawdown is $D_t = P_t - E_t$ and max drawdown is $\max_t D_t$. Duration is the longest consecutive sequence of steps with $D_t > 0$.

Both numbers matter. A large max drawdown with short duration is a very different problem from a moderate but persistent drawdown. Depth without duration hides fatigue risk in strategies with slow recovery.

5.4 Rolling information ratio

The viewer computes a rolling information ratio over a window of w steps:

$$\mathrm{IR}_w = \frac{\mathbb{E}_w[\Delta E_t]}{\mathrm{sd}_w(\Delta E_t)} \cdot \sqrt{w}.$$

This is a diagnostic ratio over the simulation window. It is **not** annualised to calendar time and should not be compared to annualised Sharpe ratios from real strategies. The simulation covers several hundred simulated seconds of a stylised instrument - annualising it would be numerically meaningless.

5.5 HFT-specific diagnostics

Time at the inside. The fraction of steps where the MM's resting bid equals the best bid (and similarly for the ask). High inside share with poor P&L is a signal of adverse selection. Low inside share with good P&L suggests the spread is unnecessarily wide.

Queue position at fill. At each fill, record the maker's index in the FIFO queue at that level. The distribution of queue positions is a proxy for fill quality. A high mean index means the agent is posting late or behind large resting size. Earlier posting or smaller size allocation at crowded levels may improve the mix.

Markout by horizon. For each MM fill, compute

$$\mathrm{markout}_H = \mathrm{sign} \times (\mathrm{fill_px} - \mathrm{mid}(t + H)),$$

where sign $= +1$ if the taker bought from the MM ask (positive if mid falls after) and sign $= -1$ if the taker sold into the MM bid (positive if mid rises after). Positive markout at horizon H means the fill was profitable at that horizon. Tracking markout at 1, 5, 10 and 30 simulated seconds shows whether the adverse selection is short-lived or persistent.

Adverse selection stratification. Beyond aggregate markout, breaking fills down by queue position at entry, maker age at fill, volume depleted ahead before the fill, and book imbalance at fill time shows *where* the adverse selection is coming from. These are the structural variables that a desk would use to decide whether to quote more aggressively at certain queue depths.

6 The reality check: why v2.0 was a toy

The models above are mathematically coherent. The v2.0 implementation was not. Showing the v2.0 code to a core developer at HRT or Tower would result in it being binned for three specific reasons.

1. **Cache-hostile memory layout.** Using `std::list<Order>` for price-level queues means every insert is a `new` and every erase is a `delete`. The allocator overhead is 50-100 ns per operation, and because each list node lives at a random heap address, iterating a queue during matching is a sequence of independent cache misses. `std::map<price_t, Level>` compounds this: $O(\log N)$ per level access, with each step a pointer hop into a cold cache line. For a book with 20 active levels, that is 4-5 pointer chases per lookup.

2. **Constant latency is not a model.** A fixed cancel latency of 30 ms assumes every cancel arrives after exactly 30 ms. Real co-location RTT has a modal value near the hardware floor and a right tail from kernel scheduler pre-emption, TCP retransmit events, and exchange-side garbage collection pauses. Constant latency lets the strategy optimistically cancel every stale quote as if jitter does not exist.

3. **The agent was using information it cannot have.** The v2.0 agent priced its quotes around the latent OU fundamental. A real market maker does not have access to a noise-free fundamental signal. It derives fair value entirely from observable book data. Letting the agent peek at the fundamental is not a simplification - it breaks the information asymmetry that drives adverse selection, which is the most important dynamic to understand.

7 The v3.0 engine: production-oriented architecture

7.1 OrderPool: slab allocator with intrusive lists

There are zero heap allocations on the matching hot path in v3.0. All `Order` objects are pre-allocated in a single flat `std::vector` (the pool) of 65,536 slots. Each `Order` embeds `pool_prev` and `pool_next` as `uint32_t` pool indices directly in the struct, forming intrusive doubly-linked lists without any separate node objects.

Allocating an order is a free-list stack `pop_back`: approximately 2 ns. Freeing is a `push_back`. Because all orders live in contiguous DRAM, iterating a price-level queue during matching steps through addresses separated by `sizeof(Order)` bytes - the hardware prefetcher handles this well.

Cancel is $O(1)$: fetch the order by its pool index, rewire its neighbours' `pool_prev`/`pool_next` fields, and return the slot to the free list.

7.2 ArrayBook: O(1) level access

`std::map<price_t, Level>` is replaced by a power-of-2 circular slot array of 8,192 entries. The level for price P sits at slot index

$$(P - \text{base})\ \&\ \texttt{kMask},$$

where `kMask` = 8191. This is a single array lookup with no branching and no pointer chasing. Best-bid and best-ask tracking use a `std::set<price_t>`, which is $O(\log k)$ for insert and erase, but k is the number of live price levels (typically under 30) and updates are far less frequent than level lookups. The `begin()` call to find the best bid or ask is $O(1)$.

Each slot maintains `total_qty` incrementally - updated on every insert and match - so there is no $O(n)$ scan to compute the level depth.

Recentering (shifting the circular window when price drifts to the edge) is rare and costs $O(n_{\text{live}})$ at that moment, which is acceptable given its infrequency.

7.3 LatencyModel: log-normal tails and congestion

Cancel and place latencies are sampled from `std::lognormal_distribution` per requote cycle. The parameters are calibrated to realistic co-location RTT ranges:

Type	Mean	95th percentile	Parameters
Cancel	30 ms	≈75 ms	$\mu_{\ln} = -3.618$, $\sigma_{\ln} = 0.472$
Place	10 ms	≈23 ms	$\mu_{\ln} = -4.605$, $\sigma_{\ln} = 0.350$

The gap between the median and the 95th percentile is the operationally important number. On 5% of requote cycles the cancel arrives materially late, the market moves in the interim, and the resting quote gets lifted at a stale price. This is the mechanism the constant-latency model was hiding.

An optional congestion penalty adds proportional delay when Hawkes intensity is elevated: the matching engine is busy processing a burst, and the agent's orders queue behind it. This is disabled by default (`congestion_per_event_s = 0.0`) but can be activated to show the coupling between flow intensity and cancel success rate.

Realised latency values are written to the trade tape so the distribution can be inspected directly in the viewer.

7.4 The Hawkes flow model

Rather than the HMM described in Section 3.1, v3.0 uses a 6-dimensional Hawkes process with event types: limit add (bid and ask), cancel (bid and ask), and market order (buy and sell). The intensity of each event type at time t is

$$\lambda_i(t) = \mu_i + \sum_{j=1}^{6} \alpha_{ij} \sum_{t_k < t} e^{-\beta(t-t_k)},$$

where α_{ij} is a 6×6 asymmetric excitation matrix, and $\beta = 6$/step is the decay rate. The baseline intensities μ_i are state-dependent: they modulate with the current spread, top-of-book imbalance, and the information edge between the OU fundamental and the current mid. This encodes the intuition from the HMM - flow clusters and is directional during information events - but without requiring explicit state inference.

The row sums of α_{ij} are all below 1.0, ensuring the process is stationary. Simulation uses Ogata's thinning algorithm for exact (not discretised) event times.

7.5 Informed traders and information asymmetry

22% of market orders come from informed agents modelled in a Glosten-Milgrom style. An informed agent trades only when the OU fundamental is at least one tick away from the current mid, always in the direction of the fundamental. The MM cannot observe the fundamental and cannot distinguish informed from uninformed orders. This is the source of adverse selection in the simulation.

The OU fundamental is strictly latent: it is used only by the informed trader model. The MM agent has no access to it in `plan_quotes`.

8 The v3.0 agent: observable signals only

8.1 FairValueModel

The reservation price is derived entirely from observable book data:

$$
\begin{aligned}
\text{microprice}_t &= \frac{\text{ask}_t \cdot \text{bid_qty}_t + \text{bid}_t \cdot \text{ask_qty}_t}{\text{bid_qty}_t + \text{ask_qty}_t}, \\
\text{imb_EMA}_t &= \alpha \cdot \text{imbalance}_t + (1-\alpha) \cdot \text{imb_EMA}_{t-1}, \\
\text{fair value}_t &= \text{microprice}_t + c_{\text{imb}} \cdot \text{imb_EMA}_t, \\
\text{reservation}_t &= \text{fair value}_t - c_{\text{inv}} \cdot \text{inv}_t.
\end{aligned}
$$

Microprice (Stoikov 2018) weights the mid by the opposite-side queue depth. A large bid queue relative to the ask queue predicts that the next trade is more likely to be a market buy, so the microprice is pulled above the arithmetic mid. The imbalance EMA captures momentum: sustained one-sided book pressure shifts the fair value in the direction of the imbalance.

The imbalance EMA lives in the range $[-1, +1]$, where $+1$ means the bid is entirely dominating and -1 means the ask is entirely dominating. It is a dimensionless signal, not a price; the Streamlit viewer plots it on a separate axis from the price panels.

8.2 Toxicity-adaptive spread

A separate EMA tracks the sign of recent fills: $+1$ if the MM was the buyer (taker sold into the bid), -1 if the MM was the seller. When this EMA's magnitude exceeds a threshold, the quoted spread widens by up to `tox_widen_max_ticks`. This is the agent's observable response to the toxicity it cannot measure directly.

The requote trigger in v3.0 fires when the reservation price moves by at least one tick. In v2.x it fired when the raw mid moved. Using the reservation price is more correct: the agent responds to changes in its own assessment of fair value (incorporating book signals), not to changes in a price it may not be able to observe cleanly.

8.3 ReplayEngine

Generative models over-fit to the calibrator's assumptions. To decouple the strategy from the flow model, v3.0 includes a `ReplayEngine` that reads a normalised event CSV:

```
t_s, event, side, price_ticks, qty, order_id
```

where `event` is `A` (add limit), `C` (cancel), or `M` (market order). The replay loop drives the same `ArrayBook` and MM agent as the Hawkes generative mode. The strategy code is unchanged between modes.

`generate_synthetic()` produces test data in the identical format, so the pipeline runs without external files. To connect real data: convert a Databento MBO or Binance Level 3 feed to the CSV schema and set `cfg.use_replay = true`.

9 Streamlit viewer (v3.0)

The viewer loads the telemetry and trades CSVs and produces seven tabs.

Overview. Equity curve with drawdown, BBO and MM quotes against the OU fundamental (explicitly labelled as unobservable by the agent), inventory, P&L components, and rolling information ratio with a disclaimer that it is not annualised to calendar time.

Markout analysis. Short-horizon markout at 1, 5, 10 and 30 simulated seconds. Summary table with mean, standard deviation, median, and percentage of fills that were adversely selected at each horizon. Cumulative markout over time.

Adverse selection stratification. Markout grouped by queue position at entry, maker age at fill, volume depleted ahead before fill, and book imbalance at fill time. These four panels show the structure of adverse selection rather than just its aggregate.

Pricing signals and toxicity. Two-panel chart: microprice and BBO mid on the price axis (with the OU fundamental shown as an unobservable reference), and the imbalance EMA on a separate axis. Toxicity EMA and adaptive spread widening. Scatter of toxicity widen versus subsequent markout.

LOB depth. Stacked area chart of five-level depth on each side, top-of-book imbalance, and MM queue share at best bid and ask.

Robustness. 10-seed sweep of final equity, max drawdown, and equity versus drawdown scatter. A strategy that works on one seed and not the others tells you something.

Execution and diagnostics. Latency histogram on a **log-scale** x-axis. A linear histogram collapses the right tail into a thin sliver and makes the 50th-to-95th percentile gap invisible, which is precisely the information that matters. The panel includes a fitted log-normal overlay with percentile markers. Hawkes intensity over time. Cancel latency versus flow intensity to verify the congestion mechanism. Accounting identity residual and per-step P&L distribution.

10 Build and run

The binary is configured in `main()` and does not take command-line flags.

```
# Build
g++ -std=c++17 -O2 -o market-sim market-sim-v3.0.cpp

# Run (writes six CSVs)
./market-sim
```

The binary writes four CSVs on each run: `telemetry_v3_hawkes.csv`, `trades_v3_hawkes.csv`, `telemetry_v3_replay.csv`, `trades_v3_replay.csv`, and `multi_seed_summary_v3.csv` from the 10-seed robustness sweep.

To use real historical data, set `cfg.replay_path` to the feed CSV path and `cfg.use_replay = true` in `main()`.

```
# Launch the viewer
streamlit run market_sim_viewer_v3_0.py
```

Upload the telemetry and trades CSVs from the sidebar. The multi-seed summary is optional.

11 Summary of changes from v2.0 to v3.0

Problem in v2.x	How v3.0 fixes it
`std::list` queue nodes	Pre-allocated `OrderPool` slab; alloc/free are free-list stack operations at ≈2 ns; sequential iteration is prefetcher-friendly
`std::map` level lookup	`ArrayBook` circular slot array; level access is a bitwise mask index, $O(1)$
Constant cancel/place latency	Log-normal `LatencyModel`; mean 30 ms / 10 ms with heavy right tail; optional congestion penalty proportional to Hawkes intensity
OU fundamental as fair value	`FairValueModel` using microprice and imbalance EMA; agent is strictly blind to the OU process
Generative-only flow	`ReplayEngine` for historical L3 data; identical pipeline in both modes
Duplicate telemetry rows	Exactly one telemetry row per step; logging moved out of the fill callback
Floating-point price keys	All prices represented as `int64_t` ticks at ingress; no floating-point equality comparisons in the book

12 What I learnt

Regimes and events, not just noise. The interesting risk in market making is not the diffusive component. It is the burst - the meta-order, the liquidity shock, the news-driven sweep. The HMM framing and the Hawkes process both exist to encode that clustering.

Cache layout is part of the algorithm. The matching loop is the hot path. Using intrusive lists over a flat slab and a hash-indexed slot array instead of `std::map` is not premature optimisation. It is the difference between something that could run in a real environment and something that cannot.

Latency is a distribution. The 95th percentile cancel latency is not a curiosity. It is the rate at which your stale-quote risk materialises. Constant latency was hiding the tail entirely.

Observable signals versus assumed fundamentals. Deriving fair value from microprice and imbalance momentum forces you to trade the book in front of you. Using a latent OU process produces a cleaner backtest but a strategy that does not exist in practice.

Separate the exchange economics. Maker rebates and taker fees can flip the sign of urgent inventory flattening. If you aggregate them into P&L without tracking them separately you will draw the wrong conclusions about when to hedge.

One good historical minute beats a thousand generated paths. This is not a criticism of generative models - the Hawkes process is genuinely useful for studying dynamics in a controlled setting. But the first time you feed a real opening auction into the replay engine and watch the agent's quotes get destroyed, you understand something about market microstructure that no toy path can teach.

Appendix A: parameter reference

The values below are the defaults in `main()` and can be modified there directly.

1. **Latency.** Cancel: $\mu_{\ln} = -3.618$, $\sigma_{\ln} = 0.472$ (mean $\approx$ 27 ms, 95th $\approx$ 75 ms). Place: $\mu_{\ln} = -4.605$, $\sigma_{\ln} = 0.350$ (mean $\approx$ 10 ms). Congestion disabled by default; set `congestion_per_event_s = 1e-5` to activate.

2. **Toxicity.** EMA $\alpha = 0.30$, threshold 0.40, max widen 2 ticks. Tune against the markout stratification plots in the viewer.

3. **Fair value.** Imbalance EMA $\alpha = 0.20$, imbalance coefficient 1.5 ticks, inventory skew 0.18 ticks per share.

4. **Hawkes.** Decay $\beta = 6.0$/step. Limit baseline $\lambda = 85.0$ events/s, cancel baseline $\lambda = 38.0$, market baseline $\lambda = 12.0$. Intensity cap at 900.

5. **Informed traders.** $p_{\text{informed}} = 0.22$; minimum information edge of 1 tick before an informed order fires.

6. **Inventory threshold.** The requote band is 5 shares; set so urgent flattening occurs a few times per session, then tune.

Appendix B: formula reference

$$
\begin{aligned}
E_t &= \text{cash}_t + (m_t - \bar{p}_t) \cdot \text{inv}_t \\
\Delta E_t &= E_t - E_{t-1} \\
\text{IR}_w &= \frac{\mathbb{E}_w[\Delta E_t]}{\text{sd}_w(\Delta E_t)} \cdot \sqrt{w} \quad \text{(diagnostic ratio over simulation window; not annualised)} \\
\text{VaR}_\alpha &= \alpha\text{-quantile of } (-\Delta E_t) \\
\text{CVaR}_\alpha &= \mathbb{E}[-\Delta E_t \mid -\Delta E_t \geq \text{VaR}_\alpha] \\
\text{microprice}_t &= \frac{\text{ask}_t \cdot \text{bid_qty}_t + \text{bid}_t \cdot \text{ask_qty}_t}{\text{bid_qty}_t + \text{ask_qty}_t} \\
\text{reservation}_t &= \text{microprice}_t + c_{\text{imb}} \cdot \text{imb_EMA}_t - c_{\text{inv}} \cdot \text{inv}_t \\
\text{markout}_H &= \text{sign} \times \big(\text{fill_px} - \text{mid}(t+H)\big) \\
\text{inside share} &= \frac{\#\{t : \text{my bid} = \text{best bid}\}}{\#\text{steps}} \quad \text{(and similarly for ask)} \\
\Pi_{\text{exch}} &= f_M \cdot Q^{\text{maker}} - f_T \cdot Q^{\text{taker}} \\
\text{TOB share} &= \frac{\text{my qty at best}}{\text{level total qty}}
\end{aligned}
$$